Práctica 2
POO. Interfaces, tipos genéricos y excepciones

EI1017 – Programación Avanzada

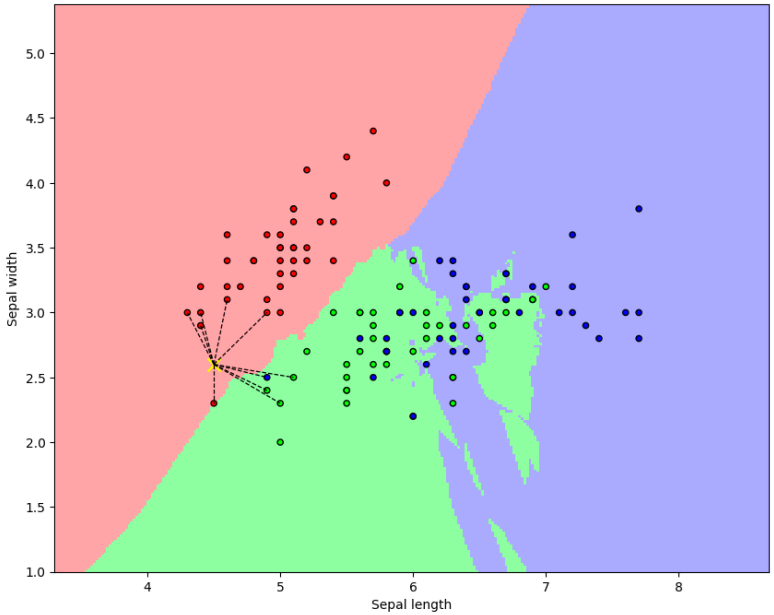
UNIVERSITAT JAUME I

Objetivos

1. Implementar el algoritmo de agrupamiento ***k-means***.
2. Definición de **interfaz genérica**.
   - Refactorización del código para implementar la interfaz.
3. Definir y lanzar **excepciones**.
4. Comprender la utilidad de las **interfaces**.

KNN (recordatorio)

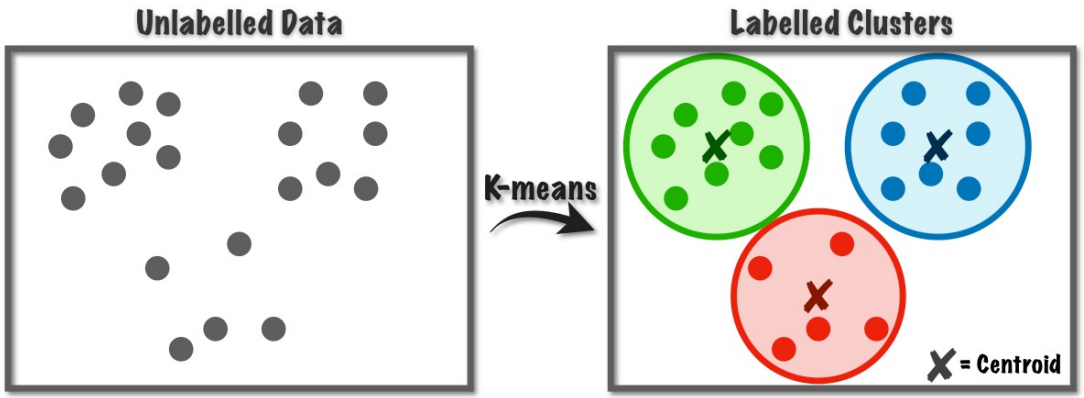
- Supervisado: Usa **datos etiquetados** para aprender (*training phase*).
- Clasificación: Determina la clase o etiqueta de nuevas muestras en base a lo aprendido (*estimation phase*).



Objetivos 1: K-Means

- No supervisado:
  - Usa **datos no etiquetados** para aprender (*training phase*).
  - **Agrupamiento** (*clustering*): se crean grupos donde sus muestras son similares entre sí y distintas a las de otros grupos.

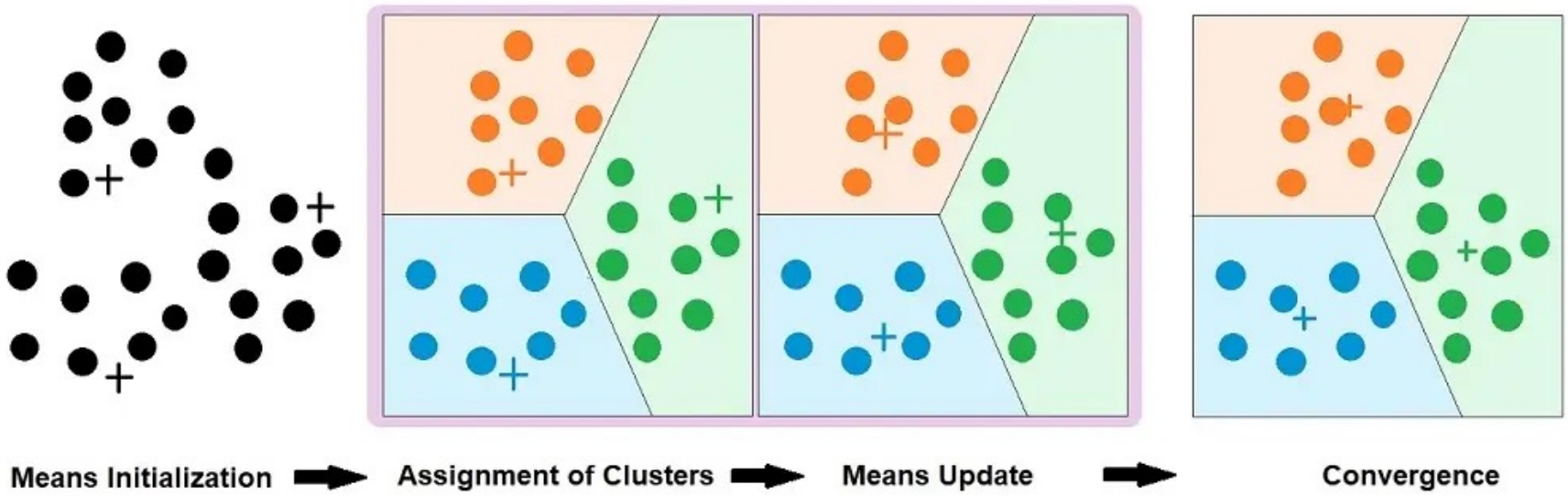
- Clasificación:
  - Determina el grupo al que pertenecen nuevas muestras en base a lo aprendido (*estimation phase*).



K-Means

- Algoritmo (*train*):
  1. Elección **aleatoria** de un representante por grupo (entre los datos de la table).
  2. Repetir *N* iteraciones:
     a) Asignar cada muestra al grupo cuyo representante está más cerca.
     b) Recalcular el centroide del grupo → Nuevo representante.
- Estimación: grupo más cercano (= representante más cercano) a la nueva muestra.

K-Means


Means Initialization
Assignment of Clusters
Means Update
Convergence

K-Means

- **Clase:**

```
public class KMeans {

    public KMeans(int numClusters, int numIterations, long seed) {
        // Almacenar número de grupos, iteraciones y semilla.
    }

    public void train(Table data) {
        // Implementación del algoritmo de entrenamiento
    }

    public Integer estimate(List<Double> sample) {

        // Implementación del algoritmo de estimación.

    }

}
```

Revisemos...


```
public class KMeans {

    public void train(Table data) {
        // ...
    }

    public Integer estimate(List<Double> sample) {

        // ...

    }
}
```

¡Ojo! Tenemos los mismos métodos en ambas clases.

```
public class KNN {

    public void train(TableWithLabels data) {
        // ...
    }

    public Integer estimate(List<Double> sample) {

        // ...

    }
}
```

Objetivos 2: Interfaz genérica

- Vamos a unificar los métodos:

1. Crear la interfaz `Algorithm`:
   - `train()` → Tipo genérico para el parámetro, `Table` o derivado.
   - `estimate()` → Tipo genérico para el parámetro y valor de retorno
2. Refactorizar `KMeans` para que implemente `Algorithm`.
3. Refactorizar `KNN` para que implemente `Algorithm`.

Objectivo 3: RecSys (utilidad de interfaz genérica)

- Vamos a emplear Algorithm para implementar un sistema de recomendación: **RecSys**.

- Idea general:
  - **Entrenamos** KNN o KMeans con ciertos datos.
  - **Inicializamos** el sistema con nuevos datos, estimando clase o grupo.
  - **Recomendación**: dado una muestra, estimamos su clase o grupo y proporcionamos otras muestras de la misma clase o grupo.

RecSys


```
public class RecSys {
    public RecSys(Algorithm algorithm) {}
    public void train(Table trainData) {}
    public void initialise(Table testData, List<String> testItemNames) {}
    public List<String> recommend(String nameLikedItem, int numRecommendations) {}
}
```

Interfaz genérica

- **`RecSys(Algorithm algorithm)`:** contruimos `RecSys` con un `Algorithm` (KNN o KMeans)

- **`train(Table trainData)`:** entrenamos el `Algorithm` con `trainData`.

UNIVERSITAT JAUME I

RecSys

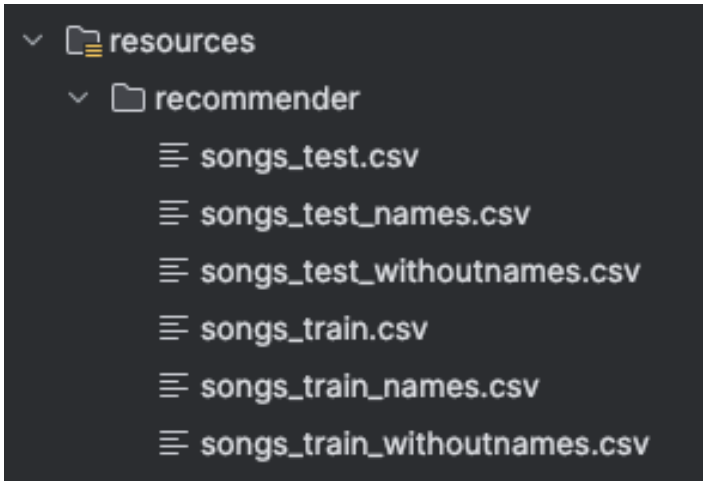
- **`initialise(Table testData, List<String> testItemNames)`**:
  - Cada muestra en `testData` tiene asociado un nombre en `testItemNames`.
  - Estimamos el grupo/clase de cada muestra en `testData` y los almacenamos.

- **`recommend(String namedLikedItem, int numRecommendations)`:**
  - Determinamos el grupo/clase asociado con `namedLikedItem`.
  - Buscamos `numRecommendations` muestras del mimso grupo/clase.
  - Devolvemos sus nombres asociados.

Usando RecSys: SongRecSys

- Usaremos `RecSys` para un sistema de recomendación de canciones: **SongRecSys**

- En el Aula Virtual tenéis todos los recursos necesarios:
  - `SongRecSys.java`
  - Ficheros CSV con información sobre canciones para entrenamiento y estimación → Almacenadlos en `resources/recommender`



Objectivo 4: Excepciones

- Crear y lanzar las siguientes excepciones:
  - `InvalidClusterNumberException` → El número de clústeres en KMeans **es mayor** a la cantidad de muestras de entrenamiento.
  - `LikedItemNotFoundException` → `nameLikedItem` pasado a `recommend` en `RecSys` **no existe** en el sistema de recomendación.

Pruebas

En el Aula Virtual os proporcionaremos pruebas en JUnit para verificar que vuestra implementación funciona correctamente.

Conclusión

Más detalles en el enunciado del Aula Virtual.

Sesiones: **17/03**, **24/03** y **31/03**

Entrega: **miércoles 8/04**

Puntuación: **1.2 puntos**

UNIVERSITAT JAUME I

¿Dudas?
UNIVERSITAT
JAUME I